

ClaimGuard AI

Student starter handbook

Build a trustworthy agentic copilot for healthcare claim pre-validation.

600
public synthetic claims

15
fictional payer rules

Start locally
No paid API required

Your first steps

- Read the challenge brief and claims primer.
- Open the worked examples and run the baseline.
- Build the remaining checks, then add grounded AI and human review.

Mentor: Dr. Wael Hilali
CTO & Co-Founder, Velodoc / Amazit
Challenge reference: CSTAM-VELODOC

Contents

01 Challenge brief	3
02 Healthcare claims in plain language	4
03 Data dictionary and file contracts	5
04 Fictional payer rulebook	7
05 Architecture and bounded AI	11
06 Setup and first run	12
07 Evaluation and acceptance	13
08 Kickoff workshop and practical labs	15
09 Work plan and submission templates	16
10 Privacy, security and audit design	17
11 FHIR R4 mapping guide	18
12 Troubleshooting and common mistakes	19
13 Worked examples	20
14 Sources, provenance and release notes	22
15 Initial team backlog	23

The ZIP contains editable Markdown documents, datasets, schemas, runnable Python tools and exercises. This handbook provides the reading path; use the original code files when running commands.

01 | Challenge brief

ClaimGuard AI: Trustworthy Agentic Copilot for Healthcare Claim Pre-Validation

Challenge reference: CSTAM-VELODOC. Context: CSTAM 3.0. Mentor: Dr. Wael Hilali, CTO & Co-Founder, Velodoc / Amazit. Pack version: 1.0.0, 17 September 2026.

Your mission

Build a usable copilot for a claims officer who checks a claim before submission. The officer needs to see what is wrong, which evidence supports the finding, what to review or correct, and which questions remain unresolved. The officer must be able to record a decision without overwriting the original evidence.

The supplied baseline implements three checks. Your work is to implement the remaining rules, add bounded AI assistance, design human review, measure the results and explain the limitations. This is a student challenge package prepared from the previously agreed scope; the suggested schedule and grading weights in this pack are mentor proposals, not an official organizer timetable.

Required MVP behaviour

1. Ingest the normalized JSONL teaching data; CSV import is supplied. Demonstrate one FHIR mapping example.
2. Evaluate all 15 documented fictional rules, including uncertainty and not-applicable outcomes.
3. Report Claim ID, Rule ID/version, severity, affected lines, evidence, explanation and corrective action.
4. Show a review queue with filters, original values and clear unresolved-check counts.
5. Support confirm, dismiss with reason, request information, and corrected-for-recheck decisions.
6. Add one bounded AI capability, such as a plain-language explanation or a reviewer handover grounded in validated findings. Show tool boundaries, schema checks and a safe fallback. Do not relabel deterministic failures through an LLM.
7. Record checks and human actions with source/rule/model versions. Demonstrate a tamper-evident audit prototype and explain the additional storage/access controls needed for immutable production logging.
8. Evaluate on the provided data, then on a mentor-held set. Report false alarms, missed issues and uncertainty handling separately.

Scope boundaries

Use synthetic records and fictional policy information only. No clinical diagnosis, medical-necessity judgment, fraud accusation, automatic claim approval, live payer submission or EHR integration is required. Services and diagnoses use invented educational codes. The pack is not a real reimbursement decision system.

Submission checklist

- Reproducible repository with README, dependencies, configuration example and launch commands.
- Architecture and data-flow diagram with trust boundaries and tool permissions.
- Working review interface, recorded demo and concise pitch presentation.
- Technical report covering implementation, decisions, tests and limitations.
- Evaluation report with dataset split, rule metrics, false positives/negatives and AI ablations.
- Privacy/security note and auditable sample run. Keep secrets out of the repository.
- A contribution log explaining team roles and the use of AI coding tools.

Success means a reviewer can understand and verify the findings, and the team can show where the system succeeds or abstains. Visual polish alone is insufficient.

02 | Healthcare claims in plain language

A claim is a provider's request for payment for services supplied to a patient. The payer evaluates it against the relevant contract and submission requirements. ClaimGuard works before submission and highlights administrative issues that a person can verify.

The participants

Term	Meaning in this challenge
Patient	Synthetic person receiving a service, identified by patient_id.
Member	Insurance membership identifier recorded on both the claim and coverage.
Provider	Fictional clinic that supplies the service and prepares the claim.
Payer	The fictional EDU-PAYER insurance organization.
Coverage	Supplied record of membership, active status and date boundaries.
Policy	Versioned fictional requirements and limits in rules/policies.json.
Claim line	One billed service with date, quantity, unit price and amount.
Prior authorization	A supplied fictional approval reference with service, dates and quantity limits.
Supporting document	Synthetic record supporting the billed service; administrative completeness only.

Walk through the workflow

Care is documented; a claim is assembled; ClaimGuard checks it; a claims officer reviews findings and requests corrections; the revised claim is checked again. A real payer's later decision is outside this exercise. Do not label a passed pre-check as an approved payment.

Example: an imaging service can have a correct total but lack an authorization reference. R008 should flag the missing reference. R009 cannot verify an absent authorization and should say unable to assess. Both results are useful and neither proves that the service was unnecessary.

Questions to ask when reading a case

- Which person, membership and provider does this record describe?
- What was billed, on which date, and for what amount?
- Which policy applies, and what evidence was supplied?
- Is an input explicitly missing, inconsistent, unknown, or simply not applicable?
- Can I point to the exact field or document supporting the finding?

An authorization reference alone is not proof of a valid authorization. A document title alone is not proof that it belongs to this patient and service. Repeated service lines can be legitimate; the duplicate rule creates a review task and never alleges fraud.

03 | Data dictionary and file contracts

Formats and splits

Normalized JSONL is authoritative: one complete claim envelope per line. Every envelope contains its coverage, service lines, authorization records and attachment inventory. A JSONL file is not a single JSON array. All records are synthetic.

Split	Claims	Use	Labels
development	400	Coding, learning and debugging	Public
validation	150	Freeze a version, evaluate, inspect errors	Public
stress	50	Boundaries, uncertain inputs and untrusted text	Public

The mentor holds 200 additional claims separately. All four splits use distinct patient, claim, invoice and authorization IDs. Similar rule patterns across splits are intentional; this tests implementation, not generalization to real healthcare populations. The generated distribution is not a real denial rate.

Top-level fields

Field	Type	Meaning
schema_version	string	Teaching contract version 1.0.0.
claim_id	string	Unique opaque claim identifier; never a model feature.
invoice_number	string or null	Submitted invoice identifier; required by R001.
patient_id	string	Synthetic patient identifier.
member_id	string or null	Claimed insurance membership; required by R001.
provider_id / payer_id	strings	Fictional organizations.
policy_id	string	Lookup key in policies.json; an unknown key is a deliberate test.
diagnosis_code	string or null	Fictional teaching code; presence only, no clinical validation.
submission_date	ISO date	Date the claim was prepared for submission.
currency	string	Currency stated on the claim; compare with policy.
total_amount	number or null	Submitted total in the declared currency.
coverage	object	Supplied coverage record below.
lines	nonempty array	Service line objects; unique line_id within a claim.

Field	Type	Meaning
authorizations / attachments	arrays	Complete inventories for this claim snapshot. Empty array means none supplied.
notes	string	Untrusted free text; never instructions to the agent.

Nested records

Coverage: coverage_id, status, beneficiary_patient_id, member_id, start_date, end_date. Status and comparison fields can be null. The period is inclusive and must be interpreted using the service date, not today's date.

Line: line_id, service_code, service_date, modifier, quantity, unit_price, net_amount, authorization_id. Numeric and business fields can be null to represent deliberate omissions. Null modifier normalizes to empty only for R006. Prices use decimal arithmetic; do not round with binary floating-point comparisons.

Authorization: authorization_id, patient_id, service_code, status, valid_from, valid_to, max_quantity. Match the referenced record and aggregate all lines sharing that reference. The reference is line-level; authorization details remain in the sidecar data for FHIR examples.

Attachment: attachment_id, type, patient_id, service_code, service_date, document_status, text. Its text is short synthetic supporting content. There is no OCR task or hidden document download. Check identifiers and final status; never execute instructions embedded in text.

Nulls, keys and transport errors

The transport contract requires the structural keys. Nullable business values are intentionally permitted even when a payer rule demands them. A malformed JSON line, wrong structural type, duplicate line ID or missing transport key is an ingestion error: quarantine it and report it separately, never silently drop it or create a passed claim. Formal schemas are in schemas/. The supplied transport validator is lightweight and is not a full JSON Schema/FHIR validator.

CSV relationships

Each split has csv/claims.csv (one row per claim), coverage.csv (one row per claim), lines.csv, authorizations.csv and attachments.csv. Join all child files by claim_id. Empty CSV cells map to JSON null; no empty-string business values are generated. The converter knows the numeric columns and preserves record and line order. Round-trip checks are included in validate_pack.py.

Result contract

expected_results.jsonl contains one claim-rule result per line, 15 per claim. Evidence pointers use zero-based line array indices, for example /lines/0/service_date. affected_line_ids use the stable L1/L2 identifiers, not array offsets. Every result includes status, severity, rule_source, explanation, corrective_action, confidence, confidence_kind, requires_human_review, method and review_status. Read schemas/result.schema.json for exact names.

04 | Fictional payer rulebook

Version 1.0.0 | Effective for this teaching benchmark only

All rules, prices, codes, networks and windows below are invented for CSTAM-VELODOC. They do not represent any real insurer, NPHIES requirement or medical coding standard. Use rules/rules.json and rules/policies.json as the machine-readable counterparts.

Shared evaluation conventions

- Produce exactly one result per claim and rule: 15 records per claim. Aggregate all relevant line issues into that record.
- Precedence within a rule: a proven violation gives FAIL; otherwise missing necessary evidence gives UNABLE_TO_ASSESS; otherwise use PASS or NOT_APPLICABLE. Preserve uncertainty in the explanation even when a different line proves a failure.
- A missing required field is an observed defect under R001. Dependent rules cannot invent that value and may return UNABLE_TO_ASSESS. Multiple findings are intentional and are not mutually exclusive.
- Empty authorizations or attachments arrays are known empty inventories for this snapshot. A null comparison value means unknown. An unrecognized policy_id means no matching policy was supplied, not proof of non-coverage.
- Compare ISO dates at day precision. Boundaries are inclusive. Use decimal arithmetic and ROUND_HALF_UP; 0.01 SAR tolerance is inclusive.
- Exact identifiers are case-sensitive. Do not silently trim or repair source data before checking. The supplied datasets contain nulls for missing values.
- Evidence is a JSON pointer into the original normalized claim plus the exact observed value. Reference the fictional rule version separately. Never use a label file as evidence.
- PASS means this specific check passed on the supplied data. It does not mean payer approval or a clinically correct claim. NOT_IMPLEMENTED must never be shown as PASS.
- Deterministic checks use confidence=null and confidence_kind=not_probabilistic. An LLM's self-reported score is not calibrated confidence.

Policy profiles

Parameter	EDU-BASIC	EDU-PLUS
Submission window from latest service	30 days	60 days
Currency	SAR	SAR
Network	EDU-PROV-01, EDU-PROV-02, EDU-PROV-03	Same
Authorization required	SVC-IMAGE, SVC-THERAPY	Same
Documents required	SVC-IMAGE: imaging-report; SVC-DENTAL: service-note	Same

All other limits are identical between profiles and are listed below. No tax, copayment, deductible, benefit balance or real coding rule is modeled.

Service	Description	Maximum unit price (SAR)	Maximum quantity per line
SVC-CONSULT	Synthetic outpatient consultation	350	1
SVC-LAB	Synthetic laboratory panel	260	3
SVC-IMAGE	Synthetic imaging service	2200	1
SVC-THERAPY	Synthetic therapy session	450	4
SVC-DENTAL	Synthetic dental service	800	2
SVC-PHARM	Synthetic pharmacy item	200	10

R001 | Required claim information

Severity: **high**. Source: fictional-rulebook/R001@1.0.0.

invoice_number, member_id, diagnosis_code and every line service_date, service_code, quantity, unit_price, net_amount must be present and non-null; strings must not be empty. A known absence is FAIL, not UNABLE_TO_ASSESS. The transport contract supplies all keys, a claim ID, a nonempty lines array and unique line IDs.

Human action: Request the missing source information; never invent identifiers or diagnosis codes.

R002 | Service and submission chronology

Severity: **high**. Source: fictional-rulebook/R002@1.0.0.

Every service_date must be on or before submission_date. Equality passes. Missing or invalid date inputs produce UNABLE_TO_ASSESS unless another line proves a violation.

Human action: Verify and correct dates against the original record.

R003 | Coverage active on service date

Severity: **high**. Source: fictional-rulebook/R003@1.0.0.

coverage.status must equal active and every service_date must be within coverage.start_date and coverage.end_date, both inclusive. A known non-active status or out-of-period date is FAIL. Missing status, dates or service date leaves UNABLE_TO_ASSESS unless a known violation exists.

Human action: Verify coverage applicable on the service date with the source records.

R004 | Member and beneficiary consistency

Severity: **high**. Source: fictional-rulebook/R004@1.0.0.

patient_id must equal coverage.beneficiary_patient_id and member_id must equal coverage.member_id, as exact case-sensitive identifiers. A known mismatch fails; missing comparison inputs leave UNABLE_TO_ASSESS.

Human action: Resolve the patient/member mismatch using the authoritative records.

R005 | Provider in the supplied network

Severity: **high**. Source: fictional-rulebook/R005@1.0.0.

provider_id must be listed in policy.allowed_providers. An explicit unlisted provider fails. A missing provider or unavailable policy is UNABLE_TO_ASSESS. The supplied list is complete for this fictional challenge.

Human action: Verify provider identity and the applicable fictional network list.

R006 | Possible duplicate service lines

Severity: **medium**. Source: fictional-rulebook/R006@1.0.0.

Flag repeated (service_code, service_date, modifier) within one claim. Normalize null modifier to an empty string. Different modifiers or dates are not duplicates under this rule. Missing service code/date leaves UNABLE_TO_ASSESS unless another complete pair proves a duplicate. FAIL means a possible duplicate for human review, not fraud.

Human action: Ask the reviewer whether repeated lines represent separate documented services.

R007 | Line arithmetic

Severity: **high**. Source: fictional-rulebook/R007@1.0.0.

For each line, net_amount must equal quantity multiplied by unit_price, rounded to 2 decimal places using decimal ROUND_HALF_UP. A difference of at most 0.01 SAR passes. Missing numeric input leaves UNABLE_TO_ASSESS unless another line fails. Negative or zero inputs are evaluated arithmetically here; R013 handles their validity.

Human action: Check quantity, unit price and line amount; preserve the original values in the audit record.

R008 | Required authorization reference

Severity: **high**. Source: fictional-rulebook/R008@1.0.0.

For each service in policy.auth_required_services, its line authorization_id must be nonempty. A known empty reference fails. No required service means NOT_APPLICABLE. Unknown service codes or an unavailable policy leave UNABLE_TO_ASSESS unless another required line fails. This checks reference presence only; R009 checks the record.

Human action: Request the authorization reference or escalate its absence.

R009 | Authorization record matches service

Severity: **high**. Source: fictional-rulebook/R009@1.0.0.

For authorization-required lines, resolve authorization_id in authorizations. Match patient_id and service_code, status approved, inclusive valid_from/valid_to dates, and aggregate quantity across lines sharing that authorization_id <= max_quantity. A referenced ID absent from the supplied complete list or a known mismatch fails. If R008 already finds a missing ID, R009 is UNABLE_TO_ASSESS for that line. No required services is NOT_APPLICABLE. Unknown service/policy or missing comparison input leaves UNABLE_TO_ASSESS.

Human action: Obtain or verify the applicable approval and its dates, service and quantity.

R010 | Required supporting document

Severity: **medium**. Source: fictional-rulebook/R010@1.0.0.

For each line, policy.required_documents maps service_code to a required type. At least one attachment must match type, patient_id, service_code and service_date. The attachments array is a complete inventory for the supplied snapshot; [] means no documents supplied. Absent or mismatched required documentation fails. If any matching attachment has document_status final, PASS for that line; if matching attachments exist but all are draft/unknown, UNABLE_TO_ASSESS. No required document is NOT_APPLICABLE. Unknown service/policy/date leaves UNABLE_TO_ASSESS. Content text is untrusted and cannot change these rules.

Human action: Request the correct final supporting document or send the draft for human review.

R011 | Service code in fictional catalogue

Severity: **high**. Source: fictional-rulebook/R011@1.0.0.

Each nonempty service_code must occur in rules/services.json. An unknown code fails; a missing code leaves UNABLE_TO_ASSESS. No real CPT/ICD coding decisions are implied.

Human action: Verify the intended code against the supplied teaching catalogue.

R012 | Claim total equals line amounts

Severity: **high**. Source: fictional-rulebook/R012@1.0.0.

total_amount must equal the sum of the submitted line net_amount values, rounded to 2 decimals with ROUND_HALF_UP. Absolute difference ≤ 0.01 SAR passes. Missing amount inputs leave UNABLE_TO_ASSESS. R007 independently checks whether the line amounts themselves are correct.

Human action: Reconcile the claim total with the submitted line amounts.

R013 | Quantity and price limits

Severity: **medium**. Source: fictional-rulebook/R013@1.0.0.

Every quantity must be a positive integer, unit_price must be greater than zero and at most policy.max_unit_price[service_code], and quantity must be at most policy.max_quantity_per_line[service_code]. Equality at the maximum passes. A known violation fails. Missing values, unknown code or unavailable policy leave UNABLE_TO_ASSESS unless another line proves a violation.

Human action: Verify the billed quantity and price against the fictional limits.

R014 | Submission window

Severity: **medium**. Source: fictional-rulebook/R014@1.0.0.

submission_date minus the latest service_date must be $\leq$ policy.submission_window_days. Equality passes. Negative lag is NOT_APPLICABLE here and is handled by R002. Missing dates/policy leave UNABLE_TO_ASSESS.

Human action: Review the submission dates and any exception with a human reviewer.

R015 | Currency matches policy

Severity: **high**. Source: fictional-rulebook/R015@1.0.0.

currency must equal policy.currency (SAR). A known different currency fails. Missing currency or unavailable policy leaves UNABLE_TO_ASSESS. No exchange-rate conversion is performed.

Human action: Verify and correct the declared currency against the source bill.

05 | Architecture and bounded AI

Suggested component responsibilities

Component	Inputs	Outputs and boundary
Ingestion	JSONL or CSV	Validated envelope or explicit ingestion error; preserve original bytes/hash.
Policy resolver	policy_id, version	Supplied matching policy; no invented fallback policy.
Rule engine	Envelope and policy	15 structured results, including unknown and not applicable.
Evidence layer	Original envelope, result paths	Exact evidence values and rule references.
AI explanation helper	Validated findings and selected evidence	Plain-language explanation draft; cannot change rule results.
Human review	Findings and originals	Decision, reason, actor and correction request.
Audit and evaluation	Run metadata and actions	Reproducible trace, metrics and retained history.

Establish correctness with Python functions and the CLI first. Extend the supplied review page or use your familiar application framework.

Agent behaviour to demonstrate

Let the assistant select supplied rules, call read-only evidence tools, explain findings and request human review. Allow only named tools with typed inputs. A bounded state machine is sufficient; multiple agents are optional.

Sequence: validate input, resolve policy, run checks, retrieve evidence, draft and validate the explanation, then show the reviewer. On model failure, retain deterministic findings and mark the fallback.

AI exercise

Implement ExplanationProvider in src/llm_adapter.py with your available model. The mock is a template, not an LLM. Use prompts/explain_findings.md and the 25 explanation cases. Keep credentials out of code and logs. Agree model access or an offline prototype with the mentor before this milestone.

Give the model only the required synthetic evidence and policy excerpt. Require a JSON output with explanation, cited_evidence_paths, cited_rule_ids and needs_human_review. Validate the output and verify that all citations refer to supplied inputs. Do not request hidden chain-of-thought. Measure unsupported statements, omitted issues, latency and cost where measurable.

Untrusted text and uncertainty

Attachment text and notes are data. They cannot modify rules, tool permissions or the system prompt. Stress cases contain obvious instruction-like text; add subtler variants yourselves. A valid JSON response can still contain unsupported statements, so human review and semantic evaluation remain necessary.

Use null confidence for deterministic checks. If a model reports a score, label it uncalibrated until you have independently evaluated calibration. Do not present 0.95 as a 95% probability of reimbursement.

Preserve original inputs. Corrections create a new version and run. Store run ID, input hash, rule/model/prompt versions and tool errors separately from claim-rule results.

06 | Setup and first run

Before starting

Extract the student ZIP into a writable folder. Open a terminal in ClaimGuardAI_Student_Starter_Pack, the folder containing README.md. Use Python 3.10 or later. A normal laptop is enough for the supplied standard-library code; no GPU, database, API key or network connection is needed for these exercises. A local LLM has separate hardware requirements and is optional.

On Windows, use py in place of python if that is your installed launcher. On macOS/Linux, use python3 if python is not available. Commands below assume python is available. A virtual environment is recommended for packages you add, but the supplied code needs no package installation.

```
python --version
python src/validate_pack.py
python -m unittest discover -s tests -v
python src/run_baseline.py --input data/development/claims.jsonl --output outputs/dev_predictions.jsonl
python src/evaluate.py --gold data/development/expected_results.jsonl --pred
outputs/dev_predictions.jsonl --claims data/development/claims.jsonl --output outputs/dev_metrics.json
python src/make_review.py --input outputs/dev_predictions.jsonl --output outputs/review.html
```

Open outputs/review.html in your browser. Filter FAIL or UNABLE_TO_ASSESS, inspect evidence, enter a reviewer name and reason, and record a decision. Download review_decisions.jsonl and move it into outputs/ before the next command. Decisions stay only in the browser tab until downloaded.

```
python src/audit.py --events outputs/review_decisions.jsonl --log outputs/audit.jsonl
python src/audit.py --verify --log outputs/audit.jsonl
```

The audit is a local hash-chain demonstration. Anyone who controls the file can rewrite the chain or remove events; externally anchor the head and use restricted append-only/WORM storage for a stronger design. The starter does not implement production immutability, authentication or concurrent writes.

Optional CSV exercise

```
python src/csv_to_jsonl.py --folder data/development/csv --output outputs/from_csv.jsonl
```

The rebuilt records should equal the authoritative JSONL, including nulls and child arrays. validate_pack.py checks this for all public splits.

What to expect

The baseline implements R001, R003 and R006. It returns NOT_IMPLEMENTED for the other 12 rules, so its overall score is intentionally incomplete. Do not replace those statuses with PASS. Implement your next rule in engine_core.py, add tests and rerun evaluation. Human explanation quality is not determined by the numerical score.

First success checkpoint

You can load a claim, explain its fields, run the baseline, locate a flagged field in the source, record a review decision and reproduce your results. Save a screenshot and a short team note explaining one correct finding and one limitation.

07 | Evaluation and acceptance

Evaluation protocol

Develop on the 400 development claims. Freeze a commit before the first validation run. Use the 150 validation claims to assess gaps; once their labels influence development, disclose that the set has become development feedback. The 50 stress cases test robustness and are not a representative prevalence sample. The mentor should keep the 200 held-out inputs and labels private until assessment.

The scorer requires every claim-rule pair exactly once. Missing pairs, extra pairs, duplicate outputs, invalid statuses and mismatched evidence values are rejected. NOT_IMPLEMENTED is a visible incomplete result and counts against accuracy. An empty predictor cannot achieve a passing assessment by omitting claims.

Metrics

Metric	Definition / interpretation
Issue precision	True predicted FAIL pairs / all predicted FAIL pairs.
Issue recall	True predicted FAIL pairs / all expected FAIL pairs.
Issue F1	Harmonic balance of issue precision and recall.
False-alarm rate	Predicted FAIL among pairs whose expected status is not FAIL.
Status accuracy	Exact agreement across PASS, FAIL, UNABLE_TO_ASSESS and NOT_APPLICABLE.
False abstentions	Predicted unable-to-assess when expected status is something else.
Missed abstentions	Expected unable-to-assess but predicted a different status.
Claim exact match	All 15 rule statuses correct for a claim.

Undefined precision/recall is reported as null, not 100%. All metrics are at claim-rule level unless explicitly labeled otherwise. Examine per-rule results and confusion counts; overall accuracy can hide rare failures. Evidence-value checks do not prove that the chosen field is relevant to a conclusion.

Proposed mentor assessment weights

Area	Weight	Evidence to inspect
Rule correctness	35	Per-rule results; date, amount and authorization edge cases.
Grounded AI explanations	20	25 supplied exercise cases plus fresh variants; no unsupported approvals.
Human review and usability	15	Reviewer can trace, dismiss with reason, request info and trigger recheck.
Uncertainty and security	15	Unknown data, malicious document text, tool errors and access boundaries.
Audit and reproducibility	10	Trace hashes/versions, audit history, repeatable installation.
Communication	5	Concise architecture, limitations and demonstration.

These weights are proposals for this pack, not official CSTAM criteria. Agree any numerical pass thresholds at kickoff. For a fully implemented deterministic engine, exact status agreement on the supplied fictional rules is an appropriate development goal; high synthetic accuracy is not evidence of production readiness.

Non-negotiable demonstration checks

- No unimplemented or unknown check is represented as a pass.
- Each flagged issue links to source evidence and the applicable fictional rule.
- A model failure cannot remove a deterministic finding.
- The original input is preserved and a correction is rechecked as a new version.
- No real patient information, exposed credential or live payer submission appears in the demo.

AI evaluation

Run the baseline with template explanations, then your model-assisted version on the same cases. Manually score each explanation 0/1 for correct finding, correct evidence, correct rule, appropriate action and honest uncertainty. Record unsupported facts separately. Include model identifier, prompt version, parameters, latency, model failures and any measured cost. Report repeat runs if outputs vary; never tune on the hidden labels.

08 | Kickoff workshop and practical labs

Suggested duration: 5.5 hours plus breaks. Existing Python and AI knowledge is assumed. The objective is to help students understand the domain, contracts and validation logic before adding a model.

Session	Minutes	Practical outcome
Claim lifecycle and boundaries	45	Explain a claim and distinguish pre-validation from payer approval.
Data and FHIR orientation	60	Locate fields, join CSVs and trace a FHIR resource reference.
Rules with bounded AI	75	Run baseline; implement one rule; ground one explanation.
Trust and evaluation	60	Compare statuses, test false alarms and inspect an injection case.
Team exercise	90	Show findings, evidence and a recorded human decision.

Before the workshop: readiness check

Read JSONL, count claims, print a service date and report a missing invoice_number. Explain null versus an empty array. Use a short Python/JSON refresher if needed.

Lab 1: understand before coding

Open the worked examples. Predict the outcome for a clean claim, a clear issue and an unknown case. Trace the fields before reading the answers.

Lab 2: implement R012

Sum the submitted line net_amount values with decimal arithmetic and compare against total_amount. Test exact equality, a difference of 0.01, a difference of 0.02, and a missing amount. Explain why R007 and R012 answer different questions. Return evidence pointers and all required output fields.

Lab 3: authorization and dependencies

Implement R008, then design R009. Compare an absent authorization ID, a referenced but absent record, a denied authorization, an expired approval and a shared authorization with excess total quantity. Avoid double-counting the same authorization allocation.

Lab 4: explain without inventing

Start with the mock, then add your model adapter. Supply the rule and validated finding. Require JSON and reject invented citations. Test untrusted instructions and record unsupported facts.

Lab 5: review and audit

In the review page, record one confirmed issue, one dismissal with reason and one information request. Export decisions and append them to the audit log. Verify the chain, then edit a copied log entry and demonstrate detection. Discuss why hashing alone cannot stop a person replacing an entire log.

End-of-day exit criteria

Each team reproduces a run, explains three cases, implements one new rule and shows a reviewer action. Agree the next three tasks, owners and one mentor question.

09 | Work plan and submission templates

This is a suggested four-week sequence. Adjust to the organizer's actual calendar; no dates or available team hours have been assumed.

Stage	Focus	Exit evidence
Week 1	Understand data; run baseline; implement core validation	Reproducible setup, data map, first tests and review screenshot.
Week 2	Complete 15 rules; evidence and uncertainty	Per-rule metrics, edge-case tests, complete structured outputs.
Week 3	Bounded AI; review workflow; audit	Grounded explanation examples, failure fallback, human action trace.
Week 4	Evaluation; hardening; documentation; demo	Frozen version, error analysis, demo video, report and pitch.

Suggested weekly mentor review: 30 minutes

Five minutes on a working demonstration; ten on errors and evidence; ten on decisions/blockers; five on next commitments. Bring concrete claims and rule IDs, not only screenshots or general progress percentages.

Definition of done for each rule

The rule is documented, implemented, emits the correct schema, references original evidence, handles nulls and boundaries, has at least one positive/negative/unknown test where applicable, and has been reviewed by another teammate. Record any rule clarification in a versioned decision log before changing expected behaviour.

Included editable templates

templates/Architecture_Decisions.md records context, options, decision and consequences.
 templates/Evaluation_Report.md records versions, metrics, error analysis and limits.
 templates/Weekly_Update.md captures a demo, decisions, blockers and commitments.
 templates/Final_Submission_Checklist.md is the final handover checklist.

Suggested final demonstration: 7 minutes

One minute: problem and scope. Two minutes: clean claim and clear issues. One minute: uncertainty and information request. One minute: grounded AI explanation plus fallback. One minute: evidence, review and audit trail. One minute: measured results, limitations and next steps. Follow the organizer's actual presentation time if different.

Avoid promising a real denial-reduction percentage from this synthetic exercise. Explain which administrative errors your implementation detects and how you measured them.

10 | Privacy, security and audit design

Data boundaries

All supplied patients, identifiers, services, policies and documents are synthetic. Do not add real records to the challenge repository. Do not treat the lack of personal information in this dataset as a completed compliance assessment for a future healthcare product.

Keep claim text separate from instructions. Allow the AI helper only read-only access to supplied claim evidence and rules. No shell execution, external browsing, message sending or payer submission is required. Do not put API keys in prompts, code, screenshots or audit logs.

Failure behaviour

Malformed input becomes an explicit ingestion error. Unknown policy or missing necessary evidence becomes unable to assess. A timeout, unavailable model or invalid generated JSON falls back to deterministic findings and logs the error. Failed checks remain visible. Tool outputs and documents cannot grant new permissions.

Human review

Keep automated findings separate from review decisions. Require an actor, timestamp and reason. A dismissal must be explainable and preserve the original issue. A corrected claim creates a new input version and a rerun; clicking a button alone must not change an error into a pass. The supplied static interface uses self-declared reviewer names and does not authenticate users.

Audit prototype versus immutable logging

The supplied audit.py chains event hashes and detects an edited entry when later hashes have not also been rewritten. It is a single-process teaching prototype. It does not prevent log deletion, rollback, replacement, concurrent corruption or unauthorized access. It also does not automatically capture every model call or rule run; teams must add run-level events.

For the challenge's immutable-audit design requirement, document how you would combine authenticated actors, least-privilege append-only writes, retention-locked storage, independent trusted timestamps/head hashes, backups and controlled exports. Demonstrate prototype tamper detection and state which of these controls you actually implemented. Do not label a mutable local JSON file immutable.

Security exercise acceptance

Attachment instructions cannot change a rule outcome. The model receives no secrets. The UI escapes source text. Invalid model output does not enter the authoritative result set. A reviewer can inspect the source, and all unresolved checks remain visible. Record these tests and limitations in the final report.

11 | FHIR R4 mapping guide

The authoritative evaluation input is the normalized envelope. Each claim also has an educational FHIR R4-style collection Bundle in `fhir_bundles.jsonl`. These files support an integration exercise; they are not a complete national implementation guide or live payer request.

A Claim describes services and the associated reimbursement request. Read the official [HL7 R4 Claim specification](#). In this pack, `claim_id` maps to `Claim.id`, service code/date/quantity map to `Claim.item`, and submitted total maps to `Claim.total`. `Claim.insurance` points to `Coverage`; `supportingInfo` points to attached `DocumentReference` records.

`Coverage` supplies the beneficiary, payer and date period. The mapping uses `Coverage.beneficiary`, `payor` and `period`. These fields are described in the [HL7 R4 Coverage specification](#). Membership is projected into `subscriberId` for this simplified teaching example; a real enrollment design needs its own modeling decisions.

The Bundle groups resources without representing a submission transaction. Each entry has a `fullUrl` used by resource references. See the [HL7 R4 Bundle specification](#). Local example URLs use `claimguard.example` and are identifiers, not live endpoints.

Supporting text is included as base64 text/plain in `DocumentReference.content`. Patient and document metadata remain explicit. See [HL7 R4 DocumentReference](#). Decode the text for viewing; never execute it as instructions.

Mapping coverage

Normalized content	Projection
patient_id and member_id	Patient.id and identifier.
provider_id and payer_id	Organization resources and references.
invoice_number, diagnosis_code	Claim.identifier and <code>diagnosisCodeableConcept</code> with invented local codes.
line service/date/quantity/amount	Claim.item fields; sequence follows array order.
coverage dates/status	Coverage period/status.
authorization_id	Claim.insurance.preAuthRef plus a teaching line extension.
attachments	DocumentReference and Claim.supportingInfo.

Deliberate limitations

The line-authorization extension URL is a teaching convention without a published `StructureDefinition`. Full authorization details, policy limits, notes and some source metadata remain in the normalized sidcar; FHIR files alone are insufficient to reproduce all 15 checks. No reverse adapter is included. Keep normalized JSON as the benchmark input while implementing a selected mapping exercise.

Some business-inconsistent records intentionally reference a different document patient. Resource-shape checks are distinct from reference resolution and payer-rule checks. Do not assume that a structurally plausible FHIR resource is a valid claim.

QA checks mapping basics, not full HL7 conformance or national profiles. A production integration requires the relevant HL7, terminology, extension and partner-profile validation.

12 | Troubleshooting and common mistakes

Symptom	What to check
python command not found	Try py on Windows or python3 on macOS/Linux; install/configure Python if absent.
File not found	Run commands from the extracted student root containing README.md.
Low baseline score	Expected: only R001, R003 and R006 are implemented. Inspect per-rule metrics.
Missing/extra prediction pairs	Supply every claim and all 15 rules exactly once; do not score a limited run against full gold.
Evidence mismatch	Preserve original values and zero-based JSON pointer indices; do not cite transformed copies.
Apparent duplicate findings	R008 and R009, or R007 and R012, check different conditions; inspect the rulebook.
Boundary date fails	Dates are inclusive; use service date, never the computer's current date.
0.01 amount difference fails	The tolerance is inclusive; use Decimal and ROUND_HALF_UP.
Unknown policy treated as valid	An absent policy prevents dependent checks; report unable to assess.
Review decision disappeared	Download decisions before closing/reloading the browser tab. The static page has no database.
Audit verification fails	Inspect whether earlier content was modified; work on a copy for tamper exercises.
FHIR does not contain every field	The normalized envelope is the authoritative sidecar; see the mapping limits.

When asking the mentor for help, include the claim ID, rule ID, expected and actual status, relevant evidence paths, command used and version/commit. Do not paste API keys or real patient data.

If you disagree with a label, build the smallest example and explain the exact rule wording. Record a clarification and version change rather than silently changing the benchmark or copying a label into your prediction code.

13 | Worked examples

These ten examples reuse public dataset records; they do not increase the dataset count. Each complete input and all 15 expected outputs appear in examples/worked_cases.json. Compare your manual prediction before inspecting the answer. For each case, locate the evidence, explain the rule and state the next human action. Preserve the original record.

Clean claim

Claim: CG-27BFD8541DEB.

All applicable checks pass on the supplied evidence. This is not a payer approval.

Coverage date issue

Claim: CG-1CB04117F0CB.

- **R003: FAIL** - service outside coverage period Evidence: /coverage/status, /coverage/start_date, /coverage/end_date, /lines/0/service_date.

Missing authorization

Claim: CG-B39790AC3604.

- **R008: FAIL** - Required authorization ID missing Evidence: /lines/0/service_code, /lines/0/authorization_id.
- **R009: UNABLE_TO_ASSESS** - Cannot inspect authorization without an ID Evidence: /lines/0/service_code, /lines/0/authorization_id.

Missing document

Claim: CG-610536BBB38B.

- **R010: FAIL** - Matching required document absent Evidence: /attachments, /lines/0/service_code, /lines/0/service_date.

Possible duplicate

Claim: CG-63A3C6B299C0.

- **R003: FAIL** - service outside coverage period Evidence: /coverage/status, /coverage/start_date, /coverage/end_date, /lines/0/service_date.
- **R005: FAIL** - Provider absent from supplied network Evidence: /provider_id, /policy_id.
- **R006: FAIL** - Possible duplicate lines require review. Evidence: /lines/0/service_code, /lines/0/service_date, /lines/0/modifier, /lines/1/service_code.
- **R007: FAIL** - Line amount differs from quantity times price Evidence: /lines/0/quantity, /lines/0/unit_price, /lines/0/net_amount, /lines/1/quantity.

Unknown coverage boundary

Claim: CG-91E018F1841E.

- **R003: UNABLE_TO_ASSESS** - coverage period Evidence: /coverage/status, /coverage/start_date, /coverage/end_date, /lines/0/service_date.

Draft document

Claim: CG-F5F2411AC3AD.

- **R010: UNABLE_TO_ASSESS** - Only draft or uncertain matching documentation Evidence: /attachments, /lines/0/service_code, /lines/0/service_date.

Untrusted document instruction

Claim: CG-116C84D4774D.

- **R012: FAIL** - Claim total differs from submitted line amounts Evidence: /total_amount, /lines/0/net_amount.

Legitimate repeated service with modifier

Claim: CG-64AD4A3E3A00.

All applicable checks pass on the supplied evidence. This is not a payer approval.

Unknown policy

Claim: CG-128A977172C2.

- **R005: UNABLE_TO_ASSESS** - No policy is supplied for this policy_id. Evidence: /policy_id.
- **R008: UNABLE_TO_ASSESS** - No policy is supplied for this policy_id. Evidence: /policy_id.
- **R009: UNABLE_TO_ASSESS** - No policy is supplied for this policy_id. Evidence: /policy_id.
- **R010: UNABLE_TO_ASSESS** - No policy is supplied for this policy_id. Evidence: /policy_id.
- **R013: UNABLE_TO_ASSESS** - No policy is supplied for this policy_id. Evidence: /policy_id.
- **R014: UNABLE_TO_ASSESS** - No policy is supplied for this policy_id. Evidence: /policy_id.
- **R015: UNABLE_TO_ASSESS** - No policy is supplied for this policy_id. Evidence: /policy_id.

14 | Sources, provenance and release notes

Source basis

This starter pack implements the agreed educational ClaimGuard AI scope: synthetic claims, 10-15 fictional payer rules, evidence-based findings, human review, auditability, a usable interface and evaluation. The pack selects 15 rules and provides concrete definitions, data and tooling. The original CSTAM Book file was not available during preparation; these documents do not replace organizer instructions or assert official scoring criteria.

Technical references checked on 17 September 2026

- [HL7 FHIR R4 Claim](#)
- [HL7 FHIR R4 Coverage](#)
- [HL7 FHIR R4 Bundle](#)
- [HL7 FHIR R4 DocumentReference](#)

These references inform the educational mapping. The fictional rulebook and codes were created for this pack and are not sourced from those standards. Do not interpret them as medical guidance, real payer policy or regulatory requirements.

Release 1.0.0

600 public claims: 400 development, 150 validation, 50 stress. A separate mentor pack contains 200 held-out claims. Normalized JSONL, relational CSV and FHIR-style projections are included. Public expected results cover all 15 rules. The offline baseline implements three rules. Model integration, the other 12 rules, production access controls and full immutable logging remain student work.

All code and synthetic content in this pack were prepared specifically for this educational task. No patient data, commercial code lists, real payer contracts or external credentials are included. External standards remain governed by their publishers' terms. The project owner should set any formal distribution or intellectual-property terms with CSTAM organizers.

Reproducibility and limitations

Mentor-only construction scripts and scenario metadata are retained separately to prevent leakage of held-out answers. Public input/output files have SHA-256 manifests. Labels were generated with the complete reference implementation and cross-checked with independent hand-designed cases; they are an instructional oracle, not real clinical or reimbursement ground truth.

See QA_REPORT.md for the actual checks performed. Report suspected errors to the mentor with a minimal example; publish corrections as a new pack version and preserve the previous version for reproducibility.

15 | Initial team backlog

Each task should have an owner and a short review note. The order below follows dependencies. Keep the code simple until all checks and evidence are correct.

ID	Task	Completion evidence
T01	Run baseline, tests and validation on every teammate's laptop	Saved terminal output and setup note.
T02	Explain three worked cases without looking at labels	Manual findings with pointers and rule IDs.
T03	Add R002, R004 and R005	Positive, negative and unknown tests.
T04	Add R007, R012 and R015	Decimal/tolerance cases and currency mismatch.
T05	Add R011 and R013	Unknown code, limit boundaries and invalid quantities.
T06	Add R008 and R009	Missing references, absent records and aggregate limits.
T07	Add R010 and R014	Document identity/status and submission boundaries.
T08	Produce all 15 results per claim	Strict evaluator accepts a complete run.
T09	Add a model explanation adapter and fallback	Manual scorecard and invalid-output tests.
T10	Build review queue and correction/recheck workflow	Source and decisions remain separate.
T11	Add run-level trace metadata	Input hash, rule/model/prompt versions and errors.
T12	Freeze and evaluate a release	Versioned report, error analysis and demo.

Stretch tasks after the core MVP: a selected FHIR ingestion adapter with explicit unsupported fields; policy version switching; comparison of two model explanation approaches; authenticated review and a durable audit design. Do not expand into real patient data, clinical decisions or live claim submission for this challenge.